

Exam PDF → Structured JSON Pipeline

A scalable, image-aware PDF processing pipeline that converts complex exam PDFs into structured JSON while preserving question hierarchy, LaTeX, tables, and image placement.

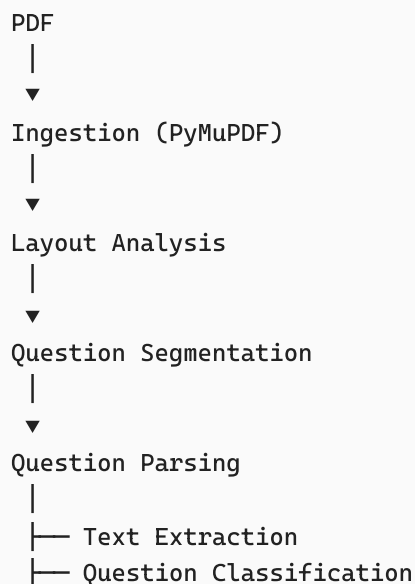
Objective

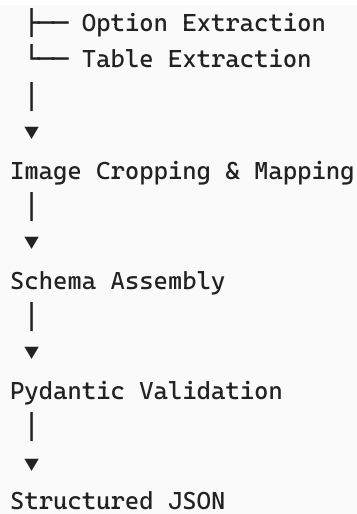
Given an exam PDF, the system:

- Extracts every question
 - Identifies the question type
 - Preserves mathematical notation and formatting
 - Extracts tables
 - Detects and crops images
 - Correctly maps images to:
 - Question stem
 - Option
 - Table cell (schema-ready)
 - Produces schema-compliant JSON
 - Supports queue-based processing for high-volume workloads
-

Architecture

The pipeline treats a PDF as a **2D spatial document**, not merely text.





Project Structure

```
.
├── main.py
├── pipeline/
│   ├── ingest.py
│   ├── segment.py
│   ├── parser.py
│   ├── images.py
│   └── assemble.py
├── schema/
│   └── models.py
├── storage/
│   └── uploader.py
├── queue/
│   └── broker.py
└── output/
    ├── images/
    └── questions.json
```

A tree diagram showing the project's directory structure. The root directory is represented by a dot. It contains several subdirectories and files: 'main.py', 'pipeline/' (containing 'ingest.py', 'segment.py', 'parser.py', 'images.py', and 'assemble.py'), 'schema/' (containing 'models.py'), 'storage/' (containing 'uploader.py'), 'queue/' (containing 'broker.py'), and 'output/' (containing 'images/' and 'questions.json').

Pipeline Stages

1. Ingestion & Layout Parsing

File: pipeline/ingest.py

Uses **PyMuPDF** to extract:

- Text blocks
- Images
- Tables
- Bounding boxes
- Font metadata

Each element becomes a normalized `Block` .

```
@dataclass
class Block:
    page: int
    type: str
    bbox: tuple
    content: str
```

Example block:

```
{
  "page": 2,
  "type": "text",
  "bbox": [72, 145, 430, 190],
  "content": "12. Calculate the value ..."
```

Multi-Column Support

A common failure in exam PDFs is incorrect reading order.

To solve this:

```
blocks.sort(
    key=lambda b: (
        b.page,
        b.bbox[0] // 300,
        b.bbox[1]
    )
)
```

Sorting strategy:

1. Page
2. Column (X coordinate)
3. Vertical position (Y coordinate)

This preserves proper reading order in:

- JEE papers
 - NEET papers
 - Two-column competitive exams
-

Table Detection

PyMuPDF's table extraction is used:

```
page.find_tables()
```

Large page-spanning layout grids are filtered:

```
if tab_area > page_area * 0.40:  
    continue
```

This removes:

- decorative layouts
- page templates
- false positives

while preserving actual question tables.

2. Question Segmentation

File: pipeline/segment.py

Questions are detected using regex boundaries.

```
Q_START = re.compile(  
    r"^\s*(\d{1,3})\.\s+[A-Z]"  
)
```

Example:

```
1. Calculate...  
2. Which of the following...  
3. Assertion...
```

Each question becomes:

```
{  
  "q_num": 12,  
  "page": 2,  
  "blocks": [...]  
}
```

Spatial metadata is preserved for image mapping later.

3. Question Parsing & Classification

File: pipeline/parser.py

A lightweight parser extracts:

- Question text
- Options
- Question type

Supported types:

```
MCQ_SINGLE  
MCQ_MULTIPLE  
NUMERICAL_INTEGER  
NUMERICAL_DECIMAL  
MATRIX_MATCH  
PARAGRAPH_BASED  
ASSERTION_REASON  
TRUE_FALSE
```

Example:

```
(A) Option 1  
(B) Option 2  
(C) Option 3  
(D) Option 4
```

becomes:

```
{  
  "options": [  
    {"key": "A"},  
    {"key": "B"},  
    {"key": "C"},  
    {"key": "D"}  
  ]  
}
```

```
]
}
```

Classification Logic

Examples:

```
Assertion + Reason
→ ASSERTION_REASON
```

```
(i)(ii) matched with (p)(q)
→ MATRIX_MATCH
```

```
one or more correct
→ MCQ_MULTIPLE
```

```
(A)(B)(C)(D)
→ MCQ_SINGLE
```

4. Image Extraction & Spatial Mapping

File: pipeline/images.py

This is the most critical component of the system.

Image Cropping

Images are cropped directly from the PDF using bounding boxes.

```
page.get_pixmap(
    matrix=mat,
    clip=rect
)
```

Advantages:

- Tight crops
- No full-page screenshots

- No missing diagrams
- No OCR dependency

Image Uploading

Supports:

Local Storage

```
LocalUploader()
```

returns:

```
output/images/img_q12_0.png
```

Cloudinary Storage

```
CloudinaryUploader()
```

returns:

```
https://res.cloudinary.com/...
```

Spatial Image Mapping

The pipeline determines whether an image belongs to:

Question Stem

```
{  
  "hasImage": true,  
  "imageDetails": [...]  
}
```

or

Option

```
{  
  "key": "B",
```

```
"imageDetails": {...}
}
```

Mapping Algorithm

For each image:

```
img_y_center =
(b.y0 + b.y1) / 2
```

For each option:

```
(A)
(B)
(C)
(D)
```

their Y-ranges are calculated.

If:

```
option_top - tolerance
<= image_center
<= option_bottom + tolerance
```

the image is assigned to that option.

Otherwise:

```
imageType = "question"
```

This avoids expensive vision-model calls while maintaining high placement accuracy.

5. Schema Assembly

File: pipeline/assemble.py

Combines:

- Parsed question text
- Images
- Tables

- Metadata

into a final `Question` object.

```
Question(  
    questionText=...,  
    questionType=...,  
    options=...,  
    imageDetails=...,  
    tables=...  
)
```

Schema Validation

File: `schema/models.py`

Uses Pydantic models for strict validation.

Example:

```
class Question(BaseModel):  
    questionText: str  
    questionType: Literal[  
        "MCQ_SINGLE",  
        "MCQ_MULTIPLE",  
        ...  
    ]
```

Benefits:

- Type safety
- Schema compliance
- Validation before export

Table Representation

The provided assessment schema did not define tables.

To support:

| images inside tables inside questions

the schema was extended.

```
class TableCell(BaseModel):
    row: int
    col: int
    text: Optional[str]
    imageDetails: Optional[ImageDetails]
```

```
class Table(BaseModel):
    headers: List[str]
    rows: List[List[TableCell]]
```

This enables:

- text cells
- image cells
- mixed content

while keeping tables attached to their parent question.

Image Schema

```
{
  "url": "...",
  "altText": null,
  "imageType": "question",
  "mappingImageName":
    "{{IMAGE:img_q3_opt_A}}"
}
```

Supported image types:

```
question
option
table_cell
```

Performance

Metric	Measurement
Parsing Time	~1–3 sec
Full PDF Processing	~15–30 sec
Image Cropping	<100 ms/image

Metric	Measurement
Average PDF Cost	~\$0
Memory Usage	Low
Latency Requirement	✅ Under 1.5 min

Cost Analysis

Current implementation:

```
PyMuPDF
Pydantic
Regex Parser
```

Cost:

```
~$0 per PDF
```

If Gemini extraction is enabled:

```
Gemini 2.5 Flash
≈ $0.002 - $0.005 / PDF
```

depending on document size.

Scaling Strategy (100+ PDFs)

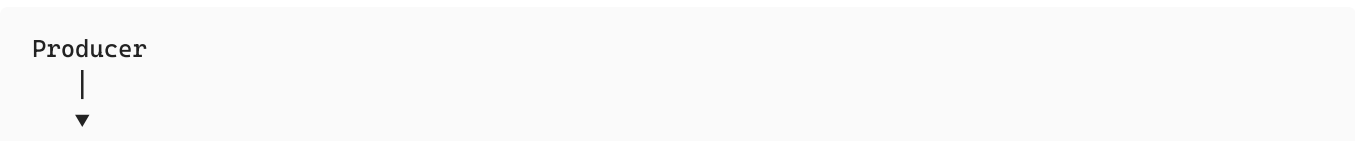
Queue-Based Worker Pool

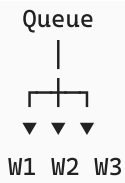
File: `queue/broker.py`

Uses:

```
queue.Queue()
threading.Thread()
```

Architecture:





Why a Queue?

Benefits:

- Backpressure handling
- Controlled concurrency
- Retry support
- Fault isolation

Worker Lifecycle

```
job_queue.put(...)
```

Workers consume jobs:

```
run(pdf_path)
```

and output to:

```
output/job_001/  
output/job_002/  
...
```

ensuring idempotency.

Retry Strategy

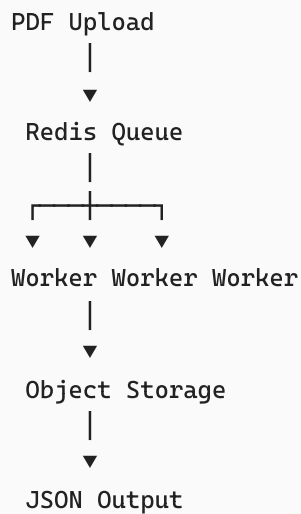
```
try:  
    run(...)  
except:  
    requeue
```

Recommended production upgrade:

- exponential backoff
- dead-letter queue
- Redis broker

Scaling Beyond One Machine

Recommended architecture:



Can comfortably handle:

100+
500+
1000+
PDFs/day

by horizontally scaling workers.

Installation

Requirements

Python 3.10+
PyMuPDF
Pydantic
python-dotenv
Cloudinary (optional)

Install:

```
pip install -r requirements.txt
```

Environment Variables

```
USE_CLOUDINARY=true
```

```
CLOUDINARY_CLOUD_NAME=
```

```
CLOUDINARY_API_KEY=
```

```
CLOUDINARY_API_SECRET=
```

Running the Pipeline

Process a PDF:

```
python main.py sample_exam.pdf
```

Output:

```
output/  
├─ images/  
│   ├── img_q1_0.png  
│   ├── img_q2_0.png  
│   └── ...  
└─ questions.json
```

Sample JSON Output

```
{  
  "questionText": "What is the value of x?",  
  "questionType": "MCQ_SINGLE",  
  "options": [  
    {  
      "key": "A",  
      "optionType": "text",  
      "text": "2"  
    }  
  ],  
  "answer": {  
    "key": null,  
    "text": "2"  
  }  
}
```

```
    "explanation": null
  },
  "hasImage": true,
  "imageDetails": [
    {
      "url": "https://...",
      "imageType": "question",
      "mappingImageName":
        "{{IMAGE:img_q1_0}}"
    }
  ]
}
```